

The Corporate Heist - Solution Documentation

****Team name:**** Neural Knights

****Author:**** Rajnish kumar

1. Summary (5-8 lines)

Our candidate shortlisting pipeline is engineered using Scikit-Learn's HistGradientBoosting model to accurately score and rank candidates for "The Corporate Heist". The solution dynamically aligns feature schemas between historical training data and current test data to eliminate structural drift. Categorical features are encoded using OrdinalEncoder with robust unseen-category handling. Predictions are converted into relative suitability scores, sorted in descending order, and the top 500 candidates are assigned strict sequence ranks (1 to 500). The output format was thoroughly validated using check_format.py to guarantee error-free submission compliance.

2. Data cleaning and parsing

- ****Schema Alignment:**** test.csv contained additional feature columns (e.g., public_code_contributions) not present in train.csv. We dynamically filtered feature matrices to retain only common valid predictor columns present in both datasets.
- ****Categorical Handling:**** String features contained diverse text formats and potential unseen categories. We passed string conversions through OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1) to prevent encoding crashes.
- ****Missing Value Management:**** Missing/null entries were kept intact for HistGradientBoosting, which natively handles missing values during tree splits without risky imputation bias.

3. What drives a great hire - your findings

- ****Pattern Verification:**** Feature importance evaluation revealed that quantitative domain capability metrics and activity scores strongly

correlated with top target outcomes in the historical training set.

- **Validation:** Verified candidate distributions by evaluating predicted probability score spreads across test records, ensuring top ranks reflect clear high-confidence predictions rather than noise.

4. The current cycle

- **Data Drift / Schema Mismatch:** The current vault extract (test.csv) introduced schema variations and feature distribution shifts compared to historical records.
- **Mitigation Strategy:** Our pipeline accounts for this by dynamically matching feature sets (X_{train} and X_{test} intersection) and using L2-regularized gradient boosted decision trees ($l2_regularization=1.0$, $max_depth=6$) to prevent overfitting on cycle-specific artifacts.

5. Profiles you excluded and why

- **Non-Top 500 Candidates:** Only the top 500 highest-scoring candidates were shortlisted based on model prediction scores to satisfy the strict submission constraint ($K=500$).
- **Outliers & Low Scores:** Candidates with low predicted suitability scores or missing critical feature signatures naturally fell below the rank 500 threshold and were excluded.

6. Model and selection procedure

- **Model:** HistGradientBoostingClassifier (or HistGradientBoostingRegressor depending on target cardinality).
- **Hyperparameters:** $max_iter=300$, $learning_rate=0.05$, $max_depth=6$, $l2_regularization=1.0$, $random_state=42$.
- **Validation Approach:** Monitored baseline format compliance via `check_format.py` and optimized tree learning rates to prevent overfitting.
- **Selection & Ranking:** Predicted continuous target probabilities/scores for all candidate IDs in test.csv, sorted the

records in descending score order, extracted the top 500, and mapped sequential integers 1 to 500 to the rank column alongside candidate_id.

7. What did not work

- **Direct Feature Matrix Fitting:** Fitting the model directly on train.csv columns without cross-referencing test.csv columns failed due to extra unseen columns (public_code_contributions KeyError/ValueError).
- **Basic Random Forest without Categorical Encoding:** Standard tree models failed or crashed when encountering unencoded string features and missing values.

8. External resources and AI tools (mandatory)

- **Libraries:** pandas (data loading/manipulation), numpy (array ops), scikit-learn (HistGradientBoosting, OrdinalEncoder for modeling).
- **AI Tools:** Google Gemini (used as an interactive coding assistant for environment debugging, schema reconciliation, and solution script optimization).

9. How to run

- **Exact Commands:**

```
```bash
cd corporate_heist_data
python solution.py
python check_format.py
```
```